

From Signal to Strategy

Product Strategy · Q3 2025

A decision framework for product leaders navigating market uncertainty

Section 01 · Foundations

The landscape has shifted. Here is what that means for us.

Module 02

Before we score signals, we need to agree on what a signal is.

The word is overloaded. We use it to mean anything from a customer complaint to a macro trend. This framework requires a tighter definition.

Context · Competitive Dynamics

The window for differentiation is narrowing.

Three converging forces — commoditized infrastructure, compressed release cycles, and rising customer switching costs — have reduced the average durable advantage window from 36 months to under 14. Teams that cannot identify signal from noise in that window will consistently miss timing.

Architecture · Signal Pipeline

How signals move from input to decision.

Four-stage processing pipeline – weekly cadence

```
---  
title: processing pipeline – weekly cadence  
---  
flowchart LR  
  A["Raw Signals"] --> B["Classify"]  
  B --> C["Score & Weight"]  
  C --> D["Decision Log"]  
  D -.->|"weekly retrospective"| B
```

Impact · Pilot Results

Six months of results across four product teams.

Measured against pre-framework baseline, same teams, same market conditions.

1. **73%** faster close
2. **4.2×** signal recall
3. **18** decisions logged
4. **91%** team alignment

The framework has four components.

- Signal Intake
 - Weekly structured collection across customer conversations, market data, and competitive moves. Normalized into a common schema before scoring.
- Scoring Model
 - Each signal scored on three dimensions: confidence, recency, and strategic relevance. Weights are team-configurable and reviewed quarterly.
- Decision Log
 - Every decision recorded with the signals that informed it, the options considered, and the criteria applied. Feeds the calibration loop.
- Calibration Loop
 - Monthly retrospective that compares predicted outcomes to actual outcomes and adjusts scoring weights accordingly.

Code in card headers and body text.

- Signal Intake `v2.4`
 - Handles 94% of structured signals without manual intervention. Average latency: `4 min` from ingestion to scored entry.
- Scoring Model `configurable`
 - Three dimensions: confidence, recency, relevance. Default weights are `33 / 33 / 33` — adjust after your first retrospective.
- Decision Log `required`
 - Every prioritization change above `P2` must carry a logged rationale. No log, no change.
- Calibration Loop `monthly`
 - Compares predicted outcomes to actuals. First meaningful weight update happens after `2 cycles`.

Signal Intake produces three outputs.

1. Weekly Signal Brief

- A ranked list of the top 10 signals from the prior week, with confidence scores and source attribution. Distributed to product leads every Monday morning.

2. Anomaly Alerts

- Real-time flags when a signal exceeds the 2σ threshold on any dimension. Routed directly to the accountable PM with a 4-hour response SLA.

3. Monthly Signal Index

- The source of truth for the calibration loop. A complete record of all signals logged, scored, and resolved in the prior month. Required reading before each retrospective.

Two failure modes the framework is designed to prevent.

- **False signal amplification.** A single loud voice — one enterprise customer, one analyst report, one competitive announcement — dominates the decision without being weighed against the full signal set. The scoring model prevents any single source from exceeding 30% of the total signal weight in a given decision.
- **Signal hoarding.** Teams collect signals but do not log decisions, so the calibration loop has nothing to learn from. The Decision Log is a required artifact for any prioritization change above P2 severity. No log, no change.

Two intake modes for different signal types.

- Structured Intake
 - Signals with clear schema: NPS verbatims, support ticket categories, feature request volumes, win/loss notes. Ingested automatically via API connectors. Scored on arrival. Zero manual handling.
- Unstructured Intake
 - Signals without schema: field observations, conference conversations, analyst briefings, competitive demos. Require human classification before scoring. Routed to the signal owner for a 48-hour classification window.

Scoring model: before and after the calibration loop.

- **Before Calibration**

- Equal weights across all three dimensions. Confidence, recency, and relevance each contribute 33% to the final score. Simple, consistent, but blind to what your market actually rewards.

- **After Calibration**

- Weights reflect your team's historical signal accuracy. If recency has consistently been the weakest predictor for your product, it gets downweighted. The model becomes a record of what you have learned.

The shift from equal weights to calibrated weights takes two retrospective cycles — roughly 60 days from adoption.

The signal was always there. We just didn't have a system that forced us to look at it before we'd already decided.

— Head of Product, Pilot Team 3

How a decision moves through the framework.

1. Signal Logged

- *Owner classifies and submits to intake queue*

2. Scored

- *Model applies current weights, generates score*

3. Brief Published

- *Signal appears in weekly brief with rank*

4. Decision Logged

- *PM records rationale, signals, predicted outcome*

5. Retrospective

- *Outcome scored, weights updated accordingly*

What the framework does not do.

- It does not make decisions — it structures the information that humans use to decide.
- It does not replace customer discovery — it scores and routes what discovery surfaces.
- It does not work without the Decision Log — calibration requires outcome data to learn from.
- It does not guarantee alignment — it surfaces disagreement earlier, which still requires resolution.
- It does not scale down to individual feature decisions — it is designed for prioritization above P2.

Four things that must be true before you begin.

1. You have a regular prioritization cadence — at minimum monthly.
2. At least one person owns signal collection full-time or as a primary responsibility.
3. Leadership has agreed to log decisions with rationale, not just outcomes.
4. You have 90 minutes per week to run the intake and scoring process.

Calibration Result · 6-Month Pilot

- 14x
 - Return on signal investment — measured as decisions that reached the right outcome on the first attempt, versus the baseline rate before the framework was adopted.

S

Section 02

Scoring Model Deep Dive

What this section covers

The scoring model is the most configurable component. This section covers the three dimensions, how weights are set initially, and how calibration updates them over time.

1. Confidence

- How many independent sources corroborate the signal. Ranges 1–5.

2. Recency

- Time-decay applied from signal date to scoring date. Half-life is team-configurable.

3. Strategic Relevance

What Would Help Us Move Forward

Next step is a working session, not a debate.

Walk these questions with me in 60–90 minutes. The output is either a design we can execute, or a shared list of what needs more work before we commit.

Finding 01 · Structured Intake

Structured intake performed above expectations — volume and latency were not concerns.

- What worked
 - API connectors handled 94% of structured signals without manual intervention. Average scoring latency was 4 minutes from ingestion. Schema normalization held across all five connected sources.
- What required tuning
 - NPS verbatim classification had an 18% error rate in the first two weeks. Required a training pass on the classification model before accuracy reached the 92% target.

Viable as designed — NLP classification requires a 2-week warm-up period on new deployments.

Key insight works on any card-bearing layout.

- Signal Intake
 - Weekly structured collection across customer conversations, market data, and competitive moves.
- Scoring Model
 - Each signal scored on three dimensions: confidence, recency, and strategic relevance.
- Decision Log
 - Every decision recorded with the signals that informed it and the criteria applied.
- Calibration Loop
 - Monthly retrospective that compares predicted outcomes to actual outcomes.

A trailing italic-only paragraph becomes an annotation.

- Signal Intake
 - Weekly structured collection across customer conversations, market data, and competitive moves.
- Scoring Model
 - Each signal scored on three dimensions: confidence, recency, and strategic relevance.
- Decision Log
 - Every decision recorded with the signals that informed it and the criteria applied.
- Calibration Loop
 - Monthly retrospective that compares predicted outcomes to actual outcomes.

Three scoring failure modes found in the pilot.

1. Recency dominance

- High-recency noise crowding out durable signal. Teams set recency weight above 50% in the first calibration pass. Corrected by capping recency weight at 40% until two calibration cycles complete.

2. Source concentration

- Single-customer signals inflating confidence scores. One enterprise customer's verbatims represented 34% of all structured intake in month one. Corrected by adding a source-diversity floor to the scoring model.

3. Outcome misclassification

- PMs logging predicted outcomes that were too vague to score at retrospective. "Improve retention" is not scoreable. "Reduce 30-day churn from 8.2% to below 7%" is.

Four requirements every decision system must meet.

- Speed
 - Decisions must close within the window they are relevant to. Systems that add latency consume the value they exist to protect.
- Auditability
 - Every prioritization decision above a threshold must carry a traceable rationale. Required for alignment and compliance.
- Adoption
 - If the team won't use it weekly, calibration never runs and the model never improves. Ninety minutes per PM is the ceiling.
- Calibration
 - The system must improve over time. A static scoring model is a spreadsheet with extra steps.

We evaluated four intake tools against the criteria.

- Tool A · Chorus
 - Speed
 - Auditability
 - Adoption
 - Calibration
 - Strong call recording and summarization. No decision logging or calibration loop. Requires separate tooling for everything downstream of intake.
- Tool B · Productboard
 - Speed
 - Auditability
 - Adoption
 - Calibration

The four tools side by side.

Criterion	Chorus	Productboard	Notion	Sprig + Log
Speed	✓	✗	✓	✓
Auditability	✗	✓	✓	✓
Adoption	✓	✓	✗	✓
Calibration	✗	✗	✗	✓
Setup time	1 day	3–4 weeks	40+ hours	Same day

Evaluated against the same four teams and the same 90-minute weekly budget constraint.

Glossary A – F

Term	Definition
Adoption	Percentage of eligible PMs filing a Decision Log entry within 24 hours of a decision close.
Auditability	The property that any decision can be reconstructed from its inputs three months later without the original author present.
Calibration	The retrospective comparison of predicted to observed outcomes, used to score the framework's accuracy.
Connector	The integration layer between Sprig and your NPS / support platforms. Owns ingestion and tagging.
Decision Log	The append-only record of every prioritization decision, its predicted outcome, and the actual outcome at retrospective time.

Glossary P – T

Term	Definition
Predicted outcome	The author's stated expectation, recorded at decision time, used as the input to calibration.
Prioritization rhythm	The team's regular cadence (weekly, biweekly) for revisiting and re-ordering work.
Retrospective	The 30-day review meeting where logged decisions are scored against observed outcomes.
Signal	Any qualitative or quantitative input to a decision — survey response, NPS comment, support ticket, sales call note.
Sprig	The micro-survey product used by Tool D to capture qualitative signal in-product.

Applying the criteria to the tools — here is where the evidence points.

- The evidence favors Tool D
 - Sprig combined with a lightweight Decision Log meets all four criteria within the 90-minute weekly budget, reaches production in the same week it is adopted, and leaves a clean exit ramp if a better native solution emerges.
- The path is not self-executing
 - Sprig requires a connector built to your NPS and support platforms. Budget 4–6 hours of engineering time in week one. After that, zero maintenance overhead.
- The Decision Log is the hardest part
 - Not technically. Culturally. PMs need to log decisions with predicted outcomes before they close, not after. This is a habit change, not a tool change.

Two options with a connector and an explanatory note below.

- **Option A · Label**

- Body text describing the first option. Enough detail to fill the card naturally and show how the layout handles a few lines of prose.

- **Option B · Label**

- Body text describing the second option. The connector arrow between them implies direction or causality — before/after, input/output, cause/effect.

The below-note sits under the cards after a hairline rule. Use it for a single contextual sentence.

How to roll this out across your organization.

1. Pick one team and one decision type

- Start with a team that already has a regular prioritization rhythm. Apply the framework only to a single decision category for the first 30 days.

2. Log everything, decide nothing differently

- In the first month, do not change how you make decisions. Just log signals and decisions as you would have made them anyway.

3. Run your first retrospective

- At day 30, score the logged decisions against outcomes. This is where the model gets its first calibration pass.

4. Expand to a second team

- With one retrospective complete, you have evidence. Use it to onboard the second team with real data, not promises.

The six signal dimensions, what they measure, and how they are scored.

1. Confidence

- Number of independent sources corroborating the signal
- *1–5 · Auto-scored*

2. Recency

- Time-decay from signal date, configurable half-life
- *0.0–1.0 · Auto-scored*

3. Relevance

- Alignment to current strategic bets, owner-scored
- *1–5 · Manual*

4. Reach

- Number of customers or segments affected

Header And Footer

Header stays uppercase — footer renders as written.

Set `header:` and `footer:` in frontmatter for deck-level labels, or use per-slide comment directives. The header uses uppercase text-transform automatically, so you write it in any case. The footer renders exactly as written.

Implementation · Token Pipeline

The tokenization call is three lines of application code.

JavaScript · SDK v2 interface

```
import { TokenVault } from "@company/token-sdk";

const vault = new TokenVault({ keyFile: "./vault.key" });

// Tokenize at ingestion
const token = await vault.tokenize(ssn, { field: "ssn", tenant: "acme" });

// Detokenize only at point of use – every call is logged
const plaintext = await vault.detokenize(token, { requestor: "claims-svc" });
```

Before & After · Key Distribution

File-distributed keys versus vault-integrated keys.

Before · File-distributed

```
# Key material on disk – anyone with  
# filesystem access can read it  
with open('./vault.key', 'rb') as f:  
    key = f.read()  
  
cipher = AES(key)  
token = cipher.encrypt(ssn)
```

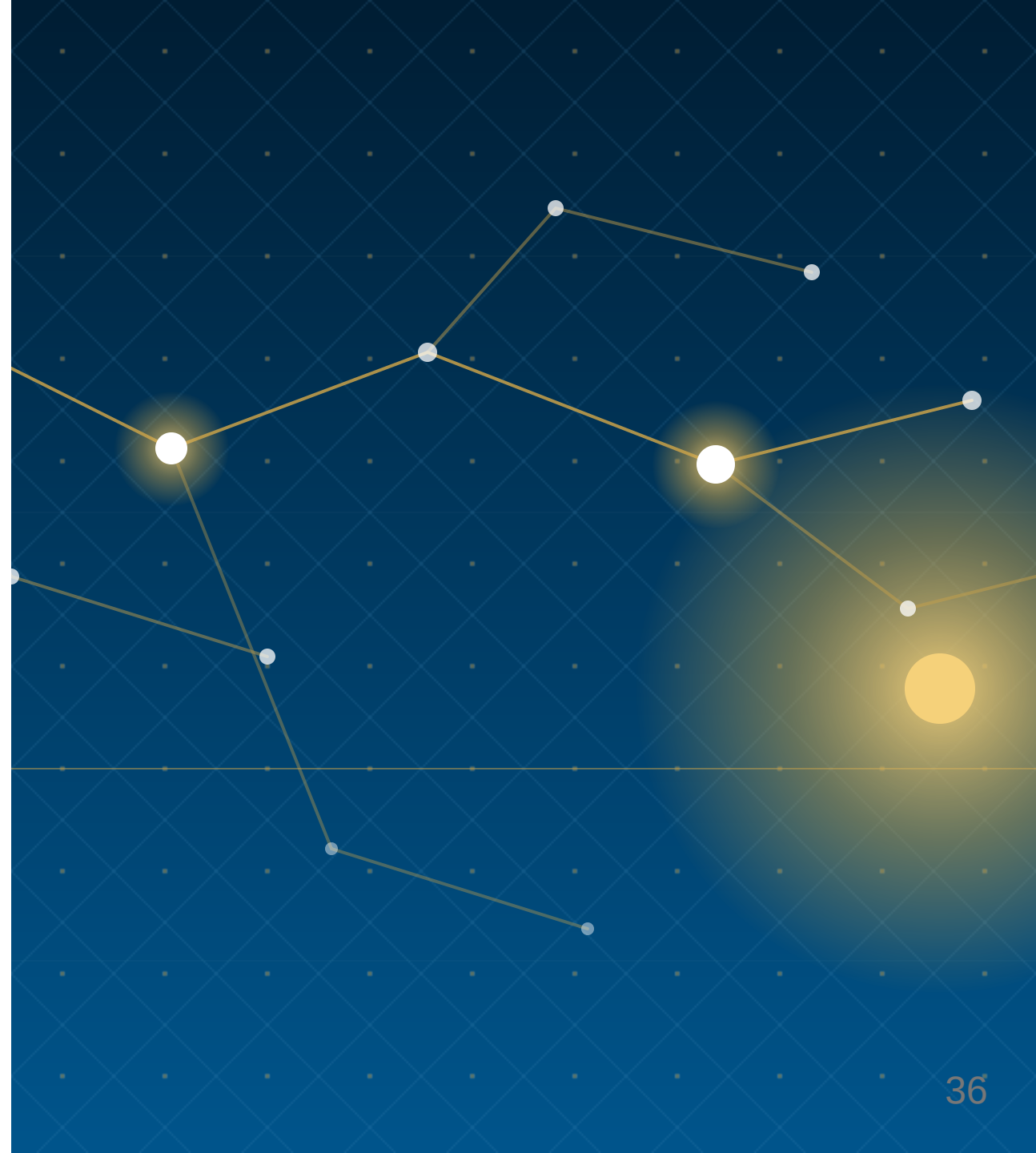
After · HSM / KMS integrated

```
# Key never leaves the HSM –  
# every operation is audited  
import boto3
```

Layout · Image

Image right is the default — text leads, evidence follows.

The image fills its half-canvas slot edge-to-edge. A 1px hairline marks the join between text and image — boardroom polish, no placeholder pattern visible behind a real photo.



Layout · Image Mirror

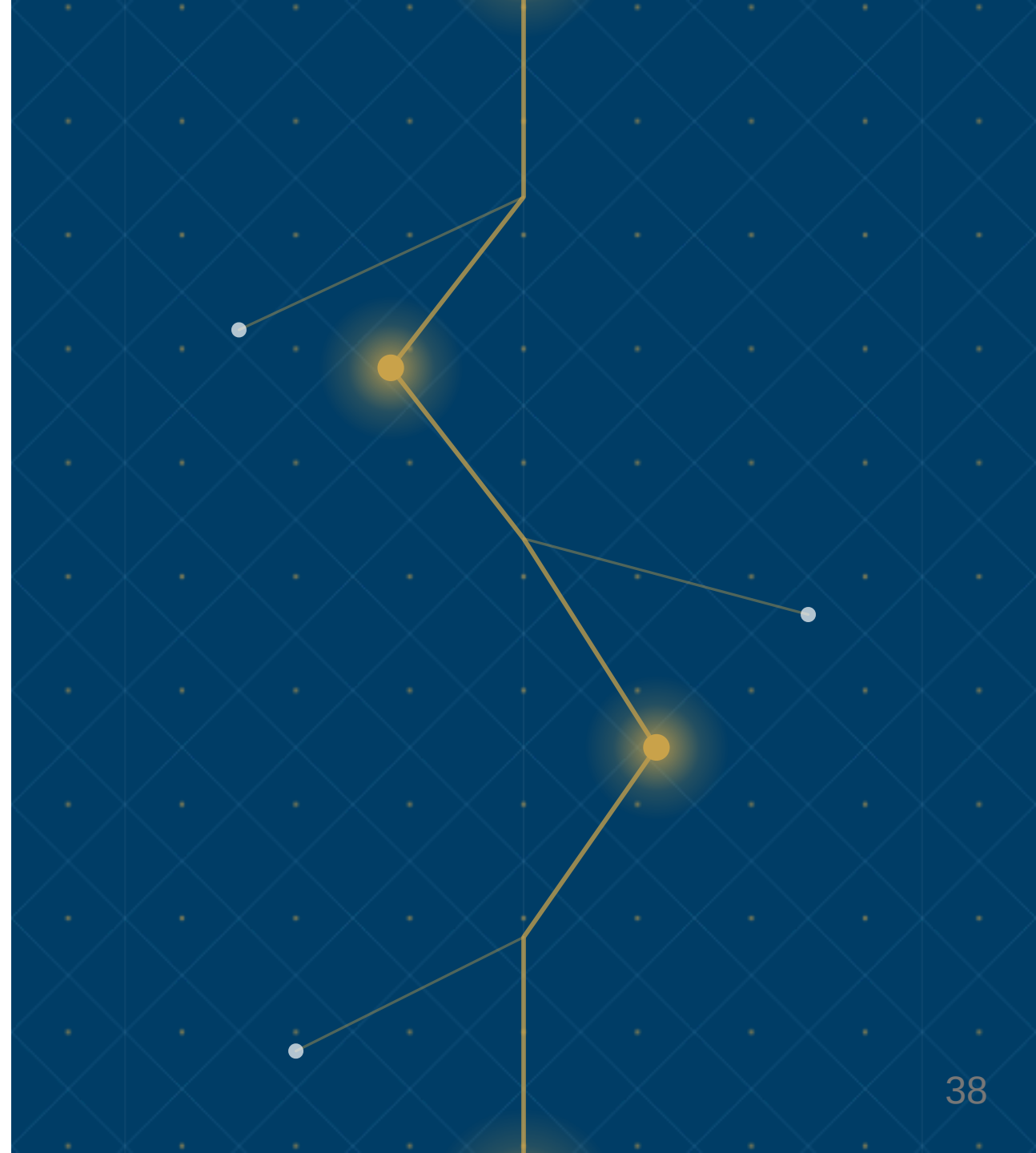
**Mirror flips the slot —
image left, text right.**

`mirror` is the cross-cutting
orientation modifier; `image left` is
preserved as a deprecated alias for
one release.

Layout · Image Contain

When a chart or screenshot must show in full, opt into `contain`.

The image is centred at native aspect on a clean `-bg-alt` matte — an editorial plate, not a placeholder. Use this for diagrams, schematics, and any asset where cropping would destroy meaning.



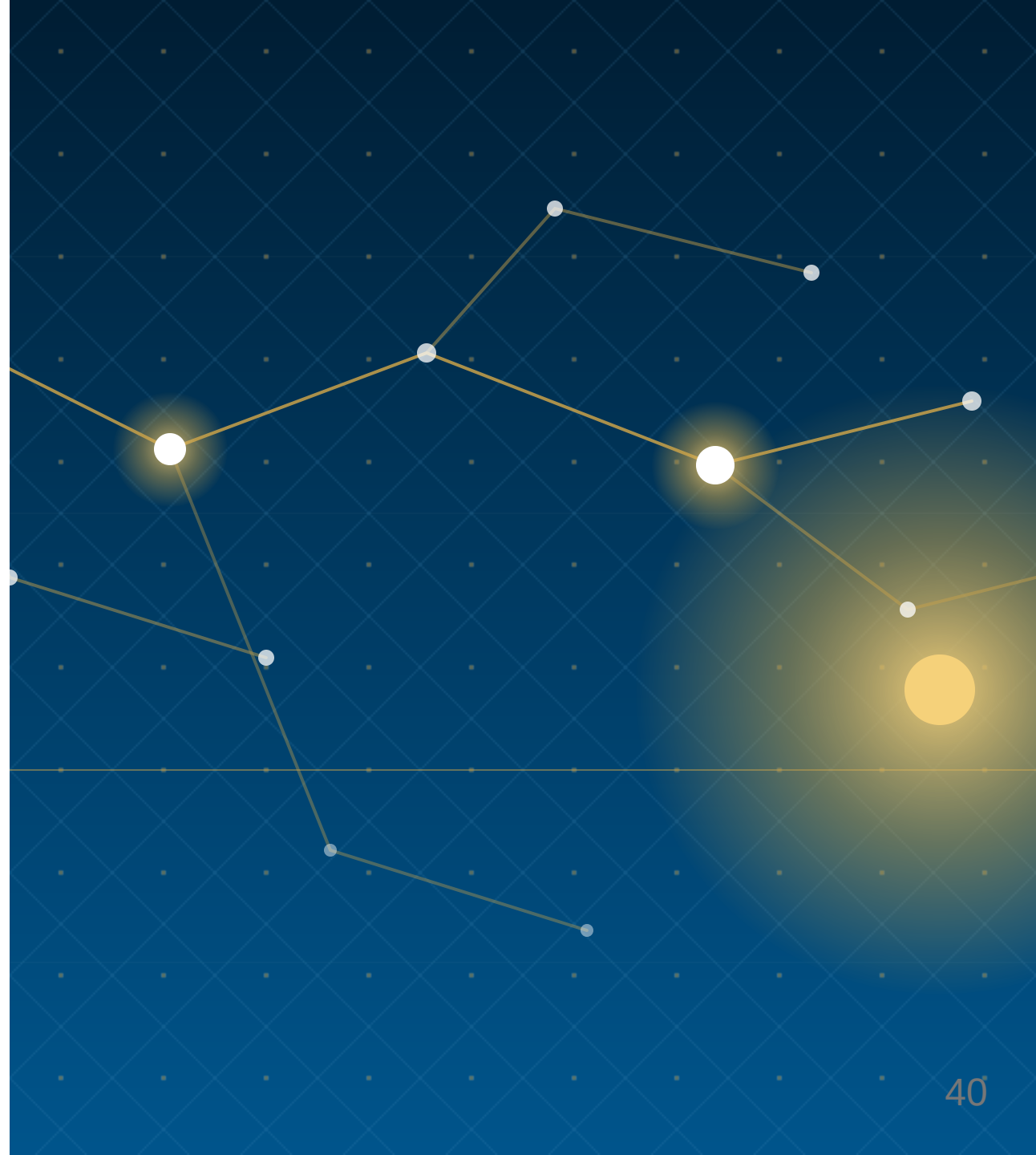
Signal Pipeline · Reference Visualization

Weekly Signal Brief — the primary output of the intake pipeline, distributed every Monday

Image Layout · Museum Modifier

Museum modifier — editorial plate

Add `museum` alongside `image`. The image panel gets a `--bg-alt` matte, 20px inset on three sides, and a 1px border. Flush at the split line, framed like a mounted print.



Architecture dependencies — every node visible

Full-bleed museum centres the asset on a generous `--bg-alt` matte with uniform 40px inset on all four sides. The border frames it as an object, not a wallpaper. No text overlaid on the image.

Dark Variant · Any Layout Class

The dark modifier works on any layout.

Add `dark` alongside any class — palette remaps automatically

Dark Variant · Content

The token system handles dark without per-element overrides.

All colours reference CSS variables — `--bg`, `--text-heading`, `--text-body`, `--border` — that remap when `dark` is added. Cards, headings, body text, and borders all shift automatically. The spectrum bar is suppressed on dark slides.

Signal Pipeline · Portrait Asset

A tall asset on a wide canvas — `contain` replaces the lattice pattern with a quiet `--bg-alt` matte, so the image reads as a museum plate rather than a placeholder.

Dark Variant · List

The card stack renders cleanly on dark backgrounds.

- Every card uses `--bg-alt` for fill and `--border` for the border — both remap in dark mode.
- The accent left border uses `--accent` which is unchanged — the gold reads well against dark.
- Body text shifts to `--text-body` which in dark mode is a warm light tone, not pure white.

Dark Variant · Cards Stacked

Two-card layouts work equally well inverted to dark.

- The architecture introduces a single key distribution question: what protects the file containing key material, and what is the blast radius if it leaves the host? Every other question in this document depends on the answer.
- The pattern here is the same as any page of written argument — claim, then support. The dark palette does not change the information density or the reading rhythm.

Modifier · list-steps phase

The phase modifier renames the prefix word from **STEP** to **PHASE**.

1. Architecture

- The first phase scopes the technical surface — what we build, what we buy, what we defer. Output is an architecture decision record signed by the platform owner.

2. Pilot

- One internal team, one workload, one quarter. The phase ends when the integration is in production and the on-call rota covers it.

3. Rollout

- Five teams in two months. The phase ends when no team needs handholding and incident volume is at or below pre-rollout baseline.

Modifier · list-steps milestone lettered

Modifiers compose: milestone renames the word, lettered swaps the format.

1. Codebook signing in production

- The HSM-anchored signing pipeline runs end-to-end. The first signed codebook installs cleanly on a real client.

2. Multi-tenant DEKs

- One codebook can carry distinct DEKs per tenant without per-tenant rebuilds. Crypto-shred is a single HSM op.

3. Per-purpose codebooks

- Authoring a codebook scoped to a single business purpose takes minutes, not days. Audit trails distinguish purposes by default.

Modifier · list-steps vertical

Vertical stacks the steps as rows; the connector becomes a down-arrow.

1. Sense

- Inputs are signals. Signals are observed, never invented. The first step is to write down what you see, not what you conclude.

2. Score

- A signal becomes data once it carries a number. The score is calibrated against outcomes, not against intuition.

3. Decide

- A decision is a signal plus a deadline. Without the deadline it is an opinion, not a decision. The retrospective closes the loop on the score that earned it.

Modifier · compare-prose chosen

Chosen flags the right-hand card as the winner.

- **Vault round-trip**
 - Every detokenize is a network call to a central vault. Latency is a function of distance, not code. p99 60 ms, vault outages cascade.
- **In-process codebook**
 - Detokenize is a local function call against an SDK-resident codebook. p99 8 ms, vault outages do not affect tokenized-record reads.

The right card carries an accent left-edge and accent-tinted background — the same visual contract used by featured cards.

Modifier · compare-prose decision

Decision composes chosen + rejected with a labelled connector.

- **Buy a vendor**
 - Three vendors evaluated; none cover the regulatory boundary in-process. Time-to-integrate is six months at best; ongoing per-tenant licensing.
- **Build in-house**
 - Owns the architecture, owns the operating model, owns the timeline. The compliance window closes in 18 months and a vendor cutover would consume nine of those.

The left card is struck through to read as the option considered then dropped; the right card carries the chosen visual; the connector is amplified and labelled DECISION.

Modifier · compare-prose vertical

Vertical stacks the two cards; the arrow connector rotates 90°.

- **Before — manual rotation**
 - Operators schedule a rotation window, freeze writes on the affected scope, swap codebooks, run a verification pass, lift the freeze. Average outage 18 minutes.
- **After — version-floor rotation**
 - The signing pipeline emits a new codebook with an incremented version. Clients install the new codebook on next refresh. No write freeze. No coordinated cutover.

Modifier · cards-grid three

Three switches the grid from 2 columns to 3 columns.

- Codebook
 - The signed envelope an SDK installs. Carries policy, wrapped DEK, version, expiry. The codebook is the unit of distribution.
- DEK
 - Data encryption key. Wrapped by a KEK; lives plaintext only inside native SDK memory. Never leaves the host.
- KEK
 - Key encryption key. Lives in the HSM, never exported. The crypto-shred operation on a tenant is a single HSM op against its KEK.

Modifier · cards-grid four

Four switches to 4 columns; pair with compact for visual balance.

- Sense
 - Signals are observed, never invented. Inputs are written down before they are interpreted.
- Score
 - A signal becomes data once it carries a number. Calibration is against outcomes, not intuition.
- Decide
 - A decision is a signal plus a deadline. Without a deadline it is an opinion.
- Review

Modifier · cards-stack horizontal

Horizontal flips cards-stack from a vertical stack to a row.

- **Claim.** The codebook model gets in-process latency with vault-grade key custody. We do not pay round-trip latency on every read.
- **Evidence.** The pilot ran six months across four product teams. p99 detokenize landed at 8 ms; vault outages did not cascade into application outages.
- **Implication.** A vendor cutover is unnecessary. We continue investing in the in-house architecture and ship the operational runbook in the next phase.

Modifier · image mirror

Mirror flips the image slot — same vocabulary as featured, split-list, compare-prose.

The half-canvas image moves from the right slot to the left, and the text padding swaps to match. `mirror` is the cross-cutting orientation flag in the Lattice grammar; `image left` is preserved as a backwards-compatible alias for one release.

Mirror puts the hero card on the right; sub-cards stack on the left.

- The hero card now reads from the right
 - The featured layout normally leads with the accented hero card on the left and stacks supporting cards on the right. Mirror swaps the columns without touching the markdown contract.
- First supporting card on the left
 - Useful when the rest of the deck reads left-to-right and the editorial weight needs to land on the right edge as the next slide opens.
- Second supporting card below
 - Identical structure, identical authoring; only the visual side changes.

S

Section 02 · Mirror

Section opener with the accent panel on the right.

What this section covers

Mirror moves the dark accent panel to the right. The watermark, eyebrow, and section number all stay anchored to the panel's own box — only the column position flips.

1. Confidence

- How many independent sources corroborate the signal. Ranges 1–5.

2. Recency

- Time-decay applied from signal date to scoring date. Half-life is team-configurable.

3. Strategic Relevance

Mirror composes with chosen — the accented card reads from the left.

- **Considered alternative**

- Source order keeps this card first, so `chosen` rules continue to target the second card in the markdown. Mirror only flips the rendering; the editorial intent (left = considered, right = chosen) is preserved by reading order.

- **The choice**

- With `mirror`, the chosen card now appears on the left visually. Use this when the surrounding deck reads right-to-left or when the chosen path needs to land first in the audience's scan path.

The below-note still appears under both cards after the hairline rule.

Modifier · divider numbered

Numbered stamps an auto-counter in the top-right corner.

The CSS counter walks the whole deck once and increments on every `divider.numbered` slide. Authors do not number sections by hand — the layout does it.

Modifier · subtopic numbered

Each bookend layout owns its own counter.

The subtopic counter is independent of the divider counter, so a mid-deck subtopic stamps 01 even when the dividers are already at 04 .

Closing · numbered

The closing series gets its own auto-stamp too.

Use it for multi-part decks where the closing slide of each part should carry the part number.

How we sort vendors against our two axes.

Coverage · Cost

- High coverage / Low cost
 - Vendor A — strongest fit on coverage, second-lowest TCO of the four.
 - Vendor B — narrower coverage but cheapest license tier.
- High coverage / High cost
 - Vendor C — full coverage, premium pricing, niche differentiators we do not need.
- Low coverage / Low cost
 - Vendor D — cheap, but leaves three regulatory boundaries uncovered.
- Low coverage / High cost
 - *none — and that is the signal.*

We are building, not buying.

Decision · 2026 Q1

- **Build**
 - Owns the architecture, owns the operating model, owns the timeline.
- **Why not buy**
 - Three vendors evaluated; none cover the regulatory boundary in-process.
- **Why not delay**
 - The compliance window closes in 18 months.

Detokenize used to require a vault round-trip.

Latency story · before vs after

- **Before**

- Every detokenize call: network round-trip to the central vault, average 18 ms, p99 60 ms. Vault outages cascaded into application outages.

- **After**

- Detokenize is a local function call. p99 8 ms. Vault outages do not affect tokenized-record reads.

The architecture change is the codebook model — local, signed, time-bound key material — not a vault optimisation.

How we make calls when the spec is silent.

1. We default to the choice that is cheaper to reverse.
2. We name the actor, never the system.
3. We write down the bet on the same slide as the choice.

What ships in each phase, by workstream.

Workstream	Phase 01	Phase 02	Phase 03
Platform	Codebook signing	Multi-tenant DEKs	Per-purpose codebooks
Operations	Manual rotation	Automated rotation	Crypto-shred
Compliance	Audit trail (HSM)	Centralised log	Examiner pack
SDK	Java		Polyglot parity

The first column is sticky workstream label; phase columns carry numbered chrome; empty cells render as a thin dash.

Where we are against quarter targets.

1. 94%

- Token-issuance success
- target 99%, +2pp QoQ

2. 8 ms

- p99 detokenize
- target 10 ms, -3 ms QoQ

3. 0

- Examiner findings
- target 0, flat

4. 3.2×

- Detokenize headroom
- target 2×, +0.4× QoQ

What this deck covers, in order.

1. The Design — page 7
2. The Phasing — page 18
3. The Choices — page 26
4. Appendices — page 35
5. Closing — page 64

Who owns each part of the codebook lifecycle.

- **Key custody** HSM admin
 - Manages KEK ceremonies and rotation. Never holds plaintext DEKs.
- **Policy** Platform operator
 - Owns codebook policy, signing keys, version floors, and revocation playbooks.
- **Consumption** Application team
 - Holds time-bound codebooks; tokenizes and detokenizes in-process.
- **Oversight** Examiner
 - Reads the HSM audit trail; cannot read plaintext.

Section 03 · Recap

What this section will tell you, in five lines.

- The codebook model gets in-process latency with vault-grade key custody. → slide 8
- Rotation is a version-floor increment, not a coordinated cutover. → slide 12
- Per-tenant KEKs make crypto-shred a single HSM op. → slide 18
- Phase 1 ships the architecture, Phase 2 ships the operations. → slide 22
- Five questions stay open until Phase 1 closes them on the record. → slide 27

Modifier · compact

Compact tightens the spacing scale ~25 %, end-to-end.

- What changes
 - `--sp-xs` through `--sp-2x1` shrink. Card gaps, list gutters, and section padding follow because every layout reads them via `var()`.
- What does not change
 - Type ramp, palette, and chrome reservation (header / footer / pagination) are untouched. Compact is a density flag, not a different layout.
- When to reach for it
 - You have one more card than fits, or your prose runs the section by 1-2 lines, or you want a denser visual rhythm without rewriting copy.
- Composition

Modifier · loose

Loose is the inverse — more breathing room, same layout machinery.

The spacing scale grows ~25 % rather than shrinks. Sections that already look generous become luxurious; sections that look cramped become balanced. Reach for `loose` when the slide carries a single editorial point and you want the page to feel deliberately quiet — values pages, declarative principles, the closing line of an argument.

The discipline is the same as `compact` from the other side: do not change the type ramp, do not change the chrome, do not change the layout. Only the variables that govern between-element rhythm move.

Modifier — loose · content loose

Density is not the same as importance. `loose` says: this page deserves room — not because it carries more, but because it carries one thing well.

Modifier · with-period

Headings gain a closing period automatically.

Authors who prefer sentence-style heading punctuation can set `class: with-period` in front matter once and stop thinking about it. The transform appends a period to any heading that does not already end with terminal punctuation — `.` `!` `?` `:` `...` — so mixed slides are safe.

The mirror modifier is `no-period`, which strips trailing periods instead. Both are deck-wide opt-ins via the global `class:` front-matter key; per-slide override with `<!--
_class: with-period -->` works too.

Modifier · no-period

Authors typed this heading with a period. It is gone

Some teams author headings with periods out of habit, then strip them in review.

`class: no-period` automates the strip so the source can stay as written and the output stays clean.

Only a literal trailing `.` is removed — `!`, `?`, `:`, and `...` pass through untouched.

Combine with any layout class; the modifier composes cleanly because it operates on the heading text alone and touches no structural chrome.

Treatment Library · Any Layout Class

tint-* and **mark-*** classes add peripheral accents from the active palette

Add a tint or mark class alongside any layout class — gradient wash or SVG mark, light canvas or dark, single pattern or layered pair.

Background · Corner Glow

A radial glow anchored at the corner fades before reaching the content zone

`tint-corner at-tl` places an elliptical accent at the top-left — 12% opacity at the corner, transparent before mid-slide. The four `at-*` placements share the same weight and fade profile; only the anchor differs.

All gradients use `color-mix(in srgb, var(--accent) 12%, transparent)`. Switching palette or adding the `dark` modifier remaps the accent automatically — no per-pattern overrides.

Background · SVG Marks · Dark

SVG accent marks are painted through a mask in the active accent colour

`mark-orbit` places concentric rings and satellite dots in the bottom-right corner. The shapes render via `::before` + `mask-image`: the SVG defines the alpha channel (white = opaque, transparent = hidden) and the paint colour is `color-mix(in srgb, var(--accent) 28%, transparent)` — resolved from the theme at render time. Same class, light canvas or dark, the shapes are always visible and always on-brand.

Background · Layered

One class from each slot layers without conflict

Every tint class (and `mark-seeds`) writes to either `--_bg-radial` or `--_bg-linear`. A compositor rule assembles both slots into a single `background-image` with two live layers. Stack one class from each column and both render:

- `tint-vignette` — radial slot — accent-tinted perimeter, open center
- `tint-edge at-right` — linear slot — wash bleeding in from the right edge

The SVG mark patterns follow the same rule: their atmospheric haze writes to its slot, and the `::before` shapes compose on top independently.

Chart Layouts · gantt + kanban

Timeline bars and board columns from a two-level list

2026 Q1 → 2026 Q4

Feature delivery by workstream

- Q1
- Q2
- Q3
- Q4
- Design
- Foundations
- Component audit
- Token refresh
- Engineering
- API v2
- SDK release
- Migration guide

Board · Phase 2 delivery

Where Phase 2 work stands today

Backlog

API contract review

Load-test harness

In progress

SDK v2 alpha in-progress

Onboarding redesign in-progress

Review

Token migration spec review

Done

Scope sign-off done

Design freeze done

Split Layouts · five variants

A structured left panel drives five two-column layouts

Q2 Performance Review

Enterprise revenue stalled in Q2

Three structural factors explain 90% of the shortfall — all addressable before Q4 close.

- **Renewal pricing complexity is driving churn at the segment ceiling**
 - Four accounts totaling \$2.1M ARR declined renewal. Win/loss interviews point to a quote-to-contract gap, not value perception.
- **Pipeline conversion dropped 11 pp below Q1 — legal review is the chokepoint**
 - Contract length increased 18 days on average. Root cause is a security addendum introduced in March.
- **Competitive displacement accelerated in the \$80–200K ACV tier**
 - Seven losses to a single competitor. Time-to-value gap is the exposure.

Net Revenue Retention

114*%*

Measured across all customers active for 12+ months, March 31 cohort.

- **Existing customers are growing faster than we lose them**
 - At 114%, every churned dollar is offset by \$1.14 in expansion. The base compounds without new-logo dependency.
- **Expansion is concentrated — three segments drive 80% of the gain**
 - Enterprise accounts in the 201–500 seat range upgrade at twice the SMB rate.
- **Sustained above 110%, this unlocks a capital-efficient growth path**
 - NRR above 110% meets the investor threshold for venture-category efficiency.

02

Discovery & Scoping

Four weeks. Shared definition of the problem before any solution work begins.

1. Stakeholder Interviews

- Eight cross-functional conversations. Open questions only — listening for friction, not confirming assumptions.

2. Current-State Audit

- System inventory, workflow documentation, and data quality review.

3. Problem Framing Workshop

- Half-day session to align on root cause. Output is a ranked problem statement the team signs off on.

4. Scope Confirmation

- Written sign-off on what is in, what is out, what requires a separate decision

Decision Required

Build the data layer or buy it?

Both paths are viable. The difference is where we spend the next 18 months.

Build in-house

- Full control over schema and roadmap
- 2–3 engineer-quarters to reach feature parity
- Ongoing maintenance burden stays internal

Buy + configure

- Ship in 6 weeks, not 9 months
- Engineering capacity redirects to product-layer features
- Exit risk manageable — data export contractually guaranteed

Buy the infrastructure. Build the differentiation. Revisit in 24 months.

The best product does not win. The most understood product does.

Morgan Chase · Head of Product, Vercel, 2024

- **Clarity is a product decision, not a marketing one**
 - If a prospect cannot articulate our value in one sentence, the product has a communication architecture problem.
- **Onboarding is the product's first argument for itself**
 - The moment a user first succeeds defines their frame for everything that follows.
- **Understanding, not delight, is the retention driver at scale**
 - Users who understand the system's logic stay through friction. Build for comprehension.

Modifier · accent

Accent replaces the rainbow stripe with a single editorial colour.

The default top border is a spectrum gradient — a system signal that the page is part of a wider deck. The `accent` modifier swaps that stripe for one solid colour and tints the slide heading. Use it when one slide carries the editorial weight of a section and you want the visual chrome to say so.

It composes with `dark`: on the dark canvas the spectrum top-stripe is suppressed entirely, so `accent.dark` restores a solid accent stripe in its place — preserving the visual signal across both canvases.